

Infraestructura como Código (IaC): Tu Primer Despliegue en Azure con Terraform (Parte 1)

Introducción

Si eres administrador de sistemas o ingeniero de infraestructura, lo más seguro es que estés acostumbrado hacerlo todo a puro clics: entrar al Portal de Azure, hacer clic en "Crear recurso", rellenar formularios infinitos y cruzar los dedos para que todo quede documentado en un Excel que nadie lee.

Hoy vamos a cambiar eso. En este artículo veremos cómo dejar atrás los procesos manuales utilizando **Terraform** para desplegar infraestructura en la nube de forma repetible, limpia y profesional. Además, lo haremos pensando en un caso de uso real: preparar el terreno para un laboratorio de red **Hub-and-Spoke** en Azure sin gastar de más.

La administración moderna de infraestructura ha evolucionado desde configuraciones manuales realizadas en portales web hacia modelos completamente automatizados basados en código.

Este enfoque recibe el nombre de Infrastructure as Code (IaC) y permite definir servidores, redes, bases de datos y recursos cloud mediante archivos de configuración reutilizables.

En este artículo aprenderás:

- Qué es IaC.
- Qué es Terraform.
- Cómo instalar Terraform en Windows.
- Cómo crear la estructura de un proyecto.
- Cómo construir tu primera plantilla.
- Cómo validar y preparar el código.
- Organización de proyectos.

Al finalizar tendrás un entorno preparado para desplegar infraestructura real en la siguiente fase utilizando Microsoft Azure.

¿Qué es Infrastructure as Code (IaC)?

Infrastructure as Code es una práctica mediante la cual la infraestructura se define mediante código en lugar de configurarse manualmente.

Administración tradicional

Un administrador realiza tareas como:

- Crear una máquina virtual.
- Crear una red.
- Configurar almacenamiento.
- Aplicar configuraciones de seguridad.

Problemas

- Difícil replicación.
- Errores humanos.
- Cambios no documentados.
- Configuraciones inconsistentes.

Administración mediante IaC

La infraestructura se describe mediante archivos:

Plain Text

main.tf

variables.tf

outputs.tf

Mostrar más líneas

Terraform interpreta estos archivos y crea automáticamente los recursos.

Beneficios

- ✓ Automatización
- ✓ Reutilización
- ✓ Versionamiento con Git
- ✓ Menor error humano

- ✓ Despliegues repetibles
- ✓ Documentación automática

¿Por qué Terraform?

Aunque Azure dispone de tecnologías nativas como ARM Templates y Bicep, Terraform destaca por su capacidad multicloud, su amplia adopción en la industria y su facilidad para gestionar infraestructura de distintos proveedores desde una misma plataforma.

¿Qué es Terraform y por qué deberías usarlo?

Terraform es una herramienta de código abierto desarrollada por HashiCorp que permite definir infraestructura utilizando un lenguaje declarativo (HCL - *HashiCorp Configuration Language*).

En lugar de hacer clics, **escribes en texto plano lo que quieres construir**. Terraform se encarga de hablar con la API del proveedor de nube (en este caso, Azure) y calcular los pasos necesarios para que tu entorno real coincida exactamente con lo que escribiste en el código.

Ventajas clave:

- **Estabilidad de resultado:** Si ejecutas el código 10 veces y la infraestructura ya existe y está correcta, Terraform no hace nada. Si algo cambia, lo corrige.
- **Control de versiones:** Tu arquitectura vive en Git. Puedes auditar cambios, hacer *rollbacks* o trabajar en equipo.
- **Destrucción limpia:** Cuando termines tus pruebas, un simple comando borra absolutamente todo, evitando sorpresas desagradables en la factura a fin de mes.

Compatibilidad

- Terraform puede trabajar con:
- Plain Text
- Azure
- AWS
- Google Cloud
- Kubernetes
- VMware
- Oracle Cloud
- GitHub
- Cloudflare

1. Instalación de Herramientas Básicas

Antes de escribir código, necesitamos preparar nuestra estación de trabajo. Solo requerimos dos herramientas esenciales (puedes instalarlas en Linux, macOS o Windows a través de WSL):

1. **Azure CLI:** Para autenticarnos contra nuestra suscripción de Azure.
2. **Terraform CLI:** El motor que procesará nuestras plantillas.

¿Qué necesitas tener?

Para la realización de este manual, se utilizó un equipo con Windows 11

- Sistema operativo: Windows 11 (arquitectura de 64 bits / AMD64, que es el estándar en la gran mayoría de equipos actuales).
- Permisos: Privilegios de Administrador en el equipo (necesarios para modificar las variables de entorno del sistema o usar gestores de paquetes).
- Terminal: PowerShell (incluido por defecto en Windows 11) o Windows Terminal.

¿De dónde se descargan los paquetes?

Nota técnica: El sitio web oficial de HashiCorp Developer

https://developer.hashicorp.com/terraform/install?utm_source=gemini. Donde se descarga el ejecutable de acuerdo a la versión de SO.

Procedimiento de Instalación

Descomprime correctamente:

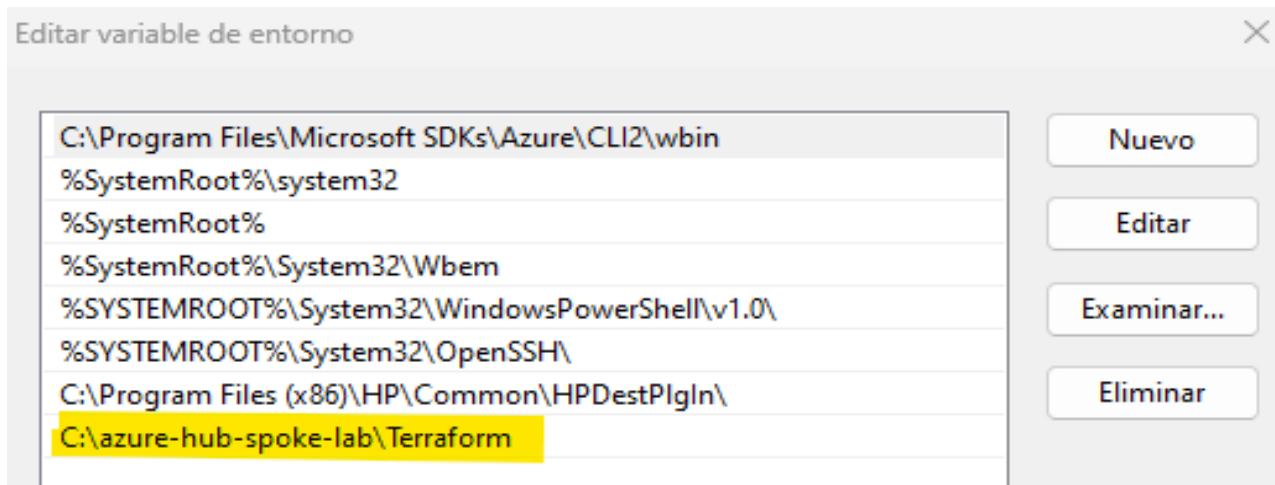
- Extrae el contenido del archivo .zip que descargaste. Asegúrate de obtener el archivo binario limpio llamado terraform.exe.

Ubícalo en una ruta fija del sistema:

- Crea una carpeta dedicada a herramientas de infraestructura, por ejemplo: C:\terraform.
- Mueve el archivo terraform.exe dentro de esa ruta. (Evita dejarlo en la carpeta Descargas, ya que las políticas de limpieza de perfil o permisos de usuario pueden romper la ruta más adelante).

Configura el PATH de Windows (Vital): Para que el sistema reconozca el comando desde cualquier terminal, debes registrar la ruta en las variables de entorno:

- Presiona Win + R, escribe sysdm.cpl y presiona Enter (esto abre las Propiedades del Sistema directamente).
- Ve a la pestaña Opciones avanzadas y haz clic en el botón Variables de entorno...
- En Variables del sistema (la sección inferior), busca y selecciona la variable Path, luego haz clic en Editar.
- Haz clic en Nuevo y añade la ruta exacta donde pusiste el ejecutable (ej. C:\terraform).
- Haz clic en Aceptar en todas las ventanas abiertas.



2. Verificar variables de entorno

Ejecutar:

```
PowerShell  
where.exe terraform
```

Alternativa mediante winget (Si prefieres evitar la gestión manual)

Si el archivo .zip te sigue dando problemas de ejecución o permisos, deja que el gestor oficial de Windows 11 maneje el despliegue de forma desatendida y limpia:

1. Abre PowerShell como Administrador.
2. Ejecuta la siguiente instrucción:

```
PowerShell
```

```
winget install --id Hashicorp.Terraform --exact
```

3. winget se encarga de descargar la arquitectura correcta, colocar el binario en su directorio seguro y registrar automáticamente las variables de entorno sin requerir interacción visual.

```
PS C:\> winget install --id Hashicorp.Terraform --exact ✓
El origen 'msstore' requiere que vea los siguientes contratos antes de usarlo.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
El origen requiere que la región geográfica de dos letras de la máquina actual se envíe al servicio back-end para que funcione correctamente (por ejemplo, "EE. UU. ").

¿Está de acuerdo con todos los términos de los contratos de origen?
[Y] Sí [N] No: y
Encontrado HashiCorp Terraform [Hashicorp.Terraform] Versión 1.16.2
El propietario de esta aplicación le concede una licencia.
Microsoft no es responsable, ni tampoco concede ninguna licencia de paquetes de terceros.
Descargando https://releases.hashicorp.com/terraform/1.16.2/terraform_1.16.2_windows_amd64.zip
34.9 MB / 34.9 MB
El hash del instalador se verificó correctamente
Extrayendo archivo...
Archivo extraído correctamente
Iniciando instalación de paquete...
Alias de línea de comandos agregado: "terraform"
Variable de entorno PATH modificada; reinicie el shell para usar el nuevo valor.
Instalado correctamente
PS C:\> terraform -version
Terraform v1.16.4 ✓
on windows_386
PS C:\>
```

Verificación de la instalación:

Abre powershell, digita el siguiente comando, el resultado será ver la versión instalada:

```
powershell
terraform -version
```

3. Estructura de Directorios Limpia (Principios SOLID y Clean Code)

Para mantener un proyecto ordenado y escalable, evita colocar todo en un solo archivo gigante. Vamos a estructurar nuestro directorio de trabajo de forma modular:

Paso 1: Crear la Estructura de Directorios

Abre tu terminal (PowerShell) y ejecuta los siguientes comandos para crear la carpeta principal del proyecto y el subdirectorio de Terraform:

```
PowerShell
mkdir C:\azure-hub-spoke-lab\terraform

cd C:\azure-hub-spoke-lab\terraform
```

Paso 2: Crear los Archivos Base

Dentro de la carpeta terraform/, vamos a crear los archivos mencionados en tu guía. Puedes crearlos usando el Bloc de notas, VS Code (code .), o mediante comandos rápidos en PowerShell.

Plaintext

```
azure-hub-spoke-lab/
├── terraform/
│   ├── provider.tf
│   ├── variables.tf
│   ├── main.tf
│   └── outputs.tf
```

Descripción de la función de los archivos creados:

provider.tf - Define los proveedores que utilizará Terraform.

variables.tf - Contiene variables reutilizables.

main.tf - Incluye los recursos principales.

outputs.tf -Muestra información útil tras el despliegue.

4. Manos a la Obra: Escribiendo el Código Base

Vamos a crear los primeros archivos de configuración dentro de tu carpeta terraform/.

Paso 1: Configurar el Proveedor (provider.tf)

Este archivo le indica a Terraform que vamos a trabajar con Azure (azurerm).

Terraform

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}
```

```
provider "azurerm" {
  features {}
}
```

Paso 2: Definir el Grupo de Recursos (main.tf)

El grupo de recursos será el contenedor lógico donde vivirá todo nuestro laboratorio de red.

Terraform

```
resource "azurerm_resource_group" "rg_lab" {
  name     = "rg-hub-spoke-lab"
  location = "eastus" # Puedes cambiarlo a "eastus2", "westus", etc., según tu preferencia}
```

5. El Flujo de Trabajo: Init, Plan y Apply

Una vez escrito tu código, el ciclo de vida de Terraform se reduce a tres comandos fundamentales:

1. **terraform init**: Inicializa el directorio, descarga el proveedor de Azure y prepara los plugins necesarios.
2. **terraform plan**: **Tu red de seguridad**. Genera una vista previa exacta de lo que Terraform va a construir en Azure. Analiza este reporte antes de aplicar cambios.
3. **terraform apply**: Ejecuta la creación real de los recursos en tu suscripción (te pedirá confirmación escribiendo yes).

Ejecutar el Flujo de Trabajo de Terraform

Ahora sí, ejecutemos el ciclo de vida estándar (Init, Plan, Apply) asegurándote de estar ubicado en la ruta C:\azure-hub-spoke-lab\terraform:

1. Inicialización (terraform.init)

Prepara el directorio y descarga los binarios del proveedor de Azure (azurerm).

```
PowerShell
terraform init
```

Salida esperada: Un mensaje indicando que Terraform has been successfully initialized! junto con una barra de progreso de descarga del proveedor.

2. Planificación (terraform.plan)

Genera la traza de ejecución previa. Esto actúa como tu red de seguridad para auditar qué cambios exactos se aplicarán en la nube.

```
PowerShell
terraform plan
```

Salida esperada: Verás un reporte detallado en color **verde** con un signo + indicando que se va a crear el recurso azurerm_resource_group_rg_lab.

3. Aplicación (terraform.apply)

Ejecuta la creación real en tu suscripción de Azure.

```
PowerShell
terraform apply
```

Comportamiento: Terraform te volverá a mostrar el plan y te pedirá una confirmación explícita escribiendo yes y presionando **Enter**.

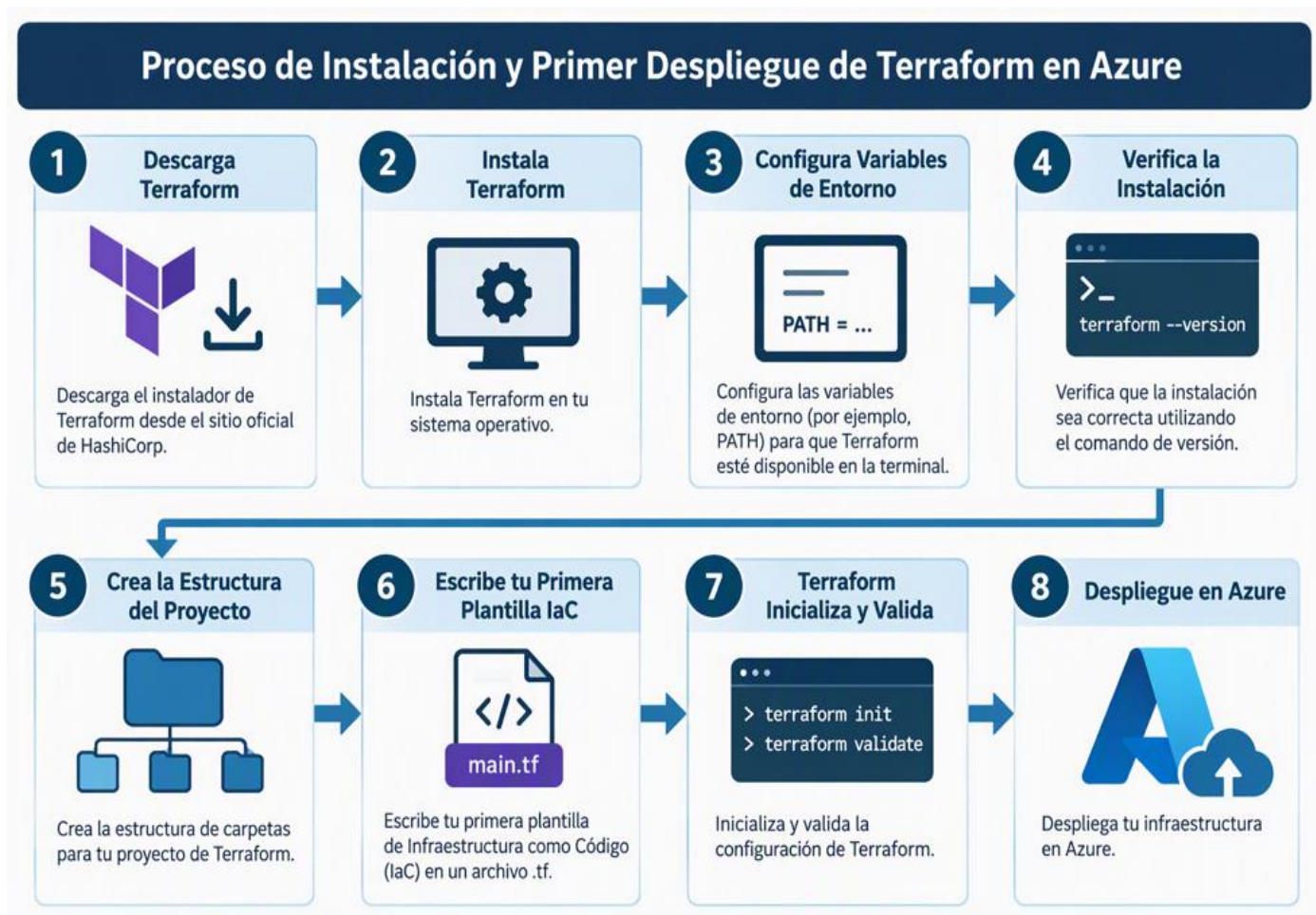
¿Qué ocurre después del despliegue?

Terraform genera un archivo llamado `terraform.tfstate`. Este archivo contiene el estado actual de la infraestructura y permite que Terraform sepa qué recursos existen y cuáles necesitan cambios en futuras ejecuciones.

¿Cómo verificar que funcionó?

Una vez finalizado el comando `apply`, puedes ir al Portal de Azure (portal.azure.com) y buscar en tus Grupos de Recursos (Resource Groups) el recurso llamado **rg-hub-spoke-lab** desplegado en la región que seleccionaste.

Flujo completo



Parametrizando tu Plantilla (Clean Code)

Para evitar quemar valores estáticos (como "rg-hub-spoke-lab" o "eastus") directamente en el código del recurso, vamos a dividirlos en sus archivos correspondientes dentro de la carpeta `terraform`.

Definir las Variables (variables.tf)

Crea un archivo llamado `variables.tf` en tu directorio. Aquí declaramos qué parámetros aceptará nuestra plantilla, añadiendo descripciones y valores por defecto limpios:

variables.tf

```
variable "resource_group_name" {
  type      = string
  description = "Nombre del grupo de recursos principal para el laboratorio."
  default    = "rg-hub-spoke-lab"
}

variable "location" {
  type      = string
  description = "Región de Azure donde se desplegarán los recursos."
  default    = "eastus"
}
```

Consumir las Variables en el Recurso (main.tf)

Modifica tu archivo `main.tf` para que ya no tenga datos "duros", sino que invoque a las variables que acabamos de crear:

main.tf

```
resource "azurerm_resource_group" "rg_lab" {
  name      = var.resource_group_name
  location  = var.location
}
```

Definir las Salidas (outputs.tf)

Crea un archivo llamado `outputs.tf`. Los *outputs* sirven para exponer información relevante de la infraestructura una vez que se despliegue (muy útil para integraciones futuras):

outputs.tf

```
output "resource_group_id" {
  description = "ID único del grupo de recursos creado en Azure."
  value       = azurerm_resource_group.rg_lab.id
}

output "resource_group_name" {
  description = "Nombre final asignado al grupo de recursos."
  value       = azurerm_resource_group.rg_lab.name
}
```

Glosario Técnico: Bloques Fundamentales en este Proyecto

En Terraform, toda la infraestructura se describe mediante archivos de configuración escritos en **HCL (HashiCorp Configuration Language)**. Para estructurar nuestro laboratorio de red de forma modular y limpia, utilizamos los siguientes componentes clave:

1. Bloque *terraform*

- **¿Qué es?:** Es el bloque de metaconfiguración raíz. Se encarga de definir los requisitos globales del proyecto.
- **Uso en este proyecto:** Lo utilizamos en `provider.tf` para obligar a que el equipo ejecute una versión mínima de Terraform ($\geq 1.5.0$) y para declarar la dependencia estricta del proveedor de Azure (`azurerm` versión ~ 3.0).

2. Bloque *provider*

- **¿Qué es?:** Es el conector o traductor entre Terraform y la API del servicio de nube con el que vamos a interactuar.
- **Uso en este proyecto:** Configura el plugin de Azure (`azurerm`). Es el encargado de recibir las credenciales de tu sesión (`az login`) y aplicar las configuraciones globales necesarias (como el bloque `features {}`).

3. Bloque *variable (Entradas)*

- **¿Qué es?:** Funciona como un parámetro o argumento de entrada. Permite desacoplar los valores estáticos (*hardcoded*) del código fuente para hacer la infraestructura reutilizable.
- **Uso en este proyecto:** Definido en `variables.tf` para parametrizar el nombre del grupo de recursos (`resource_group_name`) y la región geográfica de despliegue (`location`), permitiendo cambiar entornos sin modificar la lógica del código.

4. Bloque *resource (Recursos)* — El núcleo de la IaC

- **¿Qué es?:** Define un componente de infraestructura real que Terraform debe construir, modificar o destruir en la plataforma cloud.
- **Uso en este proyecto:** Se encuentra en `main.tf` bajo el recurso `azurerm_resource_group.rg_lab`, el cual representa el contenedor lógico donde vivirá todo el laboratorio de red.

5. Bloque *output (Salidas)*

- **¿Qué es?:** Permite extraer y mostrar información relevante después de que la infraestructura ha sido desplegada con éxito.
- **Uso en este proyecto:** Declarado en `outputs.tf` para exponer datos críticos del grupo de recursos recién creado (como su ID único o su nombre formal) para auditorías o futuras integraciones.

Fuentes Oficiales de Consulta

- Guía oficial del lenguaje (HCL): HashiCorp Developer - Terraform Language Documentation (Aquí explican a detalle cómo funciona cada bloque, tipos de datos y expresiones).
- Documentación del Proveedor de Azure: Terraform Registry - AzureRM Provider (Indispensable para buscar qué parámetros acepta cada recurso de Azure, por ejemplo, cómo configurar una red virtual o una máquina virtual).

Conclusión y Sigüientes Pasos

En esta primera parte aprendimos a preparar nuestro entorno, comprender los fundamentos de Infrastructure as Code y realizar nuestro primer despliegue utilizando Terraform.

En el siguiente laboratorio comenzaremos la construcción de una arquitectura Hub-and-Spoke en Azure, sentando las bases para escenarios más avanzados de conectividad, automatización y seguridad.

🔗 **No te pierdas la continuación de este laboratorio: En el siguiente artículo te muestro paso a paso cómo desplegar la topología completa de Hub-and-Spoke, configurar una máquina Linux como router central y optimizar tus costos para aprovechar al máximo tus créditos de prueba:**